

SILKY LAB

Manutenzione

1. Introduzione

La manutenzione del software non è solo un'attività correttiva, ma un processo strategico essenziale per il successo a lungo termine di qualsiasi applicazione. Suddividendo le attività in categorie specifiche - correttiva, preventiva, perfettiva e adattiva - si può adottare un approccio mirato e proattivo, che garantisca stabilità, innovazione e adattabilità nel tempo. Un sistema ben mantenuto non solo risponde alle criticità in maniera tempestiva, ma evolve costantemente per soddisfare le esigenze in continuo mutamento degli utenti e del mercato.

2. Tipologie di Manutenzione

- Manutenzione Correttiva

Descrizione:

Si concentra sulla risoluzione di errori e malfunzionamenti identificati durante lo sviluppo o l'utilizzo del software.

Caso Pratico:

Correzione tempestiva dei problemi riscontrati nel modulo di login o nella gestione del carrello, per ripristinare la funzionalità corretta.

- Manutenzione Preventiva

Descrizione:

Si fonda su attività proattive volte a prevenire il manifestarsi di errori futuri.

Strategie:

Esecuzione di test regolari.

Monitoraggio costante dei log per individuare anomalie prima che diventino critiche.

- Manutenzione Perfettiva

Descrizione:

Riguarda il miglioramento continuo del software, sia in termini di performance che di usabilità.

Esempio Operativo:

Introduzione di nuove funzionalità, come le notifiche per l'aggiunta di un prodotto al carrello, che rendono l'interazione più intuitiva e semplice per l'utente.

- **Manutenzione Adattiva**

Descrizione:

Interventi mirati ad adeguare il software a variazioni esterne, quali aggiornamenti di librerie o modifiche nelle normative di settore.

Scenario Tipico:

Aggiornamento delle librerie tecnologiche (ad esempio Vaadin) per garantire la compatibilità con le nuove versioni.

Nota:

Nel nostro contesto, questo tipo di manutenzione non è stato attuato, ma rappresenta una riserva da attivare in caso di cambiamenti esterni.

3. Strategie Operative per una Manutenzione Efficace

Pianificazione e Monitoraggio

- **Programmazione:** Definire un calendario di interventi per controlli e aggiornamenti periodici.
- **Monitoraggio:** Utilizzare strumenti di logging e analisi per identificare tempestivamente anomalie e inefficienze.

Implementazione delle Azioni Correttive

- **Analisi degli errori:** Identificare rapidamente le cause dei malfunzionamenti.
- **Intervento rapido:** Applicare correzioni e verificare l'efficacia attraverso test mirati.

Miglioramento Continuo

- **Feedback degli utenti:** Raccogliere e analizzare i suggerimenti per migliorare l'interfaccia e le funzionalità.
- **Aggiornamento funzionale:** Integrare nuove caratteristiche che semplifichino l'uso del software e aumentino la soddisfazione dell'utente.

Gestione delle Dipendenze Esterne

- **Adattamento alle evoluzioni tecnologiche:** Monitorare costantemente le librerie e le piattaforme esterne e prepararsi ad aggiornamenti necessari per mantenere la compatibilità e la sicurezza del sistema.
-

Refactoring

1. Valutazione Iniziale del Tangled

Attualmente l'intero progetto presenta un tangled metric pari al 22%. Questo valore, sebbene riferito all'intero sistema, è principalmente attribuibile alla classe **DatabaseManager**, che gestisce numerose responsabilità (operazioni per utenti, modelli, prodotti, indirizzi e ordini). Tale livello di complessità nelle dipendenze rappresenta un'opportunità di miglioramento per rendere il codice più modulare e facilmente manutenibile, pur non configurandosi come un problema critico.

2. Soluzioni Proposte

Per ridurre il tangled metric sono state individuate tre soluzioni:

- **Soluzione 1: Estrazione dei Metodi di Utilità**
Questa soluzione prevede lo spostamento dei metodi che manipolano le stringhe (ad esempio, la capitalizzazione di nomi e indirizzi) in una classe dedicata. Ciò permette di isolare la logica di manipolazione delle stringhe, riducendo il numero di responsabilità e semplificando la classe principale.
- **Soluzione 2: Isolamento della Gestione della Connessione**
Questa proposta consiste nel centralizzare la logica per la gestione delle connessioni al database in una classe separata. L'obiettivo è quello di delegare la responsabilità di aprire e configurare le connessioni, in modo che la classe **DatabaseManager** si concentri esclusivamente sulle operazioni sui dati.
- **Soluzione 3: Suddivisione in DAO Specifici**
La terza soluzione prevede la creazione di DAO (Data Access Object) distinti per ciascun ambito funzionale, ad esempio uno per la gestione degli utenti, uno per i modelli, uno per i prodotti, e così via. Questa strategia ridurrebbe ulteriormente il tangled dividendo la responsabilità su più classi. Tuttavia, tale approccio richiede un impegno notevole e un rifacimento più approfondito dell'architettura, motivo per cui è stato escluso in questa fase di refactoring.

3. Modifiche Applicate

In questa fase sono state implementate solo le prime due soluzioni:

- **Estrazione dei Metodi di Utilità:**

I metodi relativi alla capitalizzazione delle stringhe sono stati trasferiti in una classe dedicata, permettendo così di alleggerire la classe principale e di centralizzare la logica comune. Questo ha contribuito a ridurre le dipendenze interne, rendendo il codice più chiaro e modulare.

- **Isolamento della Gestione della Connessione:**

La logica per ottenere la connessione al database è stata spostata in una classe separata, incaricata esclusivamente di gestire l'apertura e la configurazione delle connessioni. In questo modo, la classe **DatabaseManager** ha potuto concentrarsi sulle operazioni di accesso e manipolazione dei dati, migliorando la separazione delle responsabilità.

4. Analisi Finale

Le modifiche apportate, sebbene abbiano migliorato la modularità e la chiarezza del codice, hanno portato a una riduzione del tangled metric solo in minima parte. Il valore è sceso leggermente, ma si attesta ancora intorno al 22%. È importante sottolineare che un tangled metric di questo livello non è necessariamente un problema critico, specialmente in contesti in cui la complessità intrinseca delle operazioni richiede una gestione centralizzata.

Il refactoring effettuato ha comunque:

- Incrementato la manutenibilità del codice,
- Facilitato la localizzazione e l'isolamento di logica specifica,
- Offerto una base per eventuali futuri miglioramenti, qualora si decidesse di affrontare la terza soluzione (la suddivisione in DAO specifici).

In conclusione, sebbene il valore del tangled metric non sia diminuito drasticamente, le modifiche implementate rappresentano un passo nella giusta direzione, senza impattare negativamente sul comportamento esterno della classe e garantendo una migliore struttura e organizzazione del codice.